

2026-08-19-bob126-gates

BOB-126 §11.4.263 Recommended Gate Implementation Plan

Revision: 1 **Last modified:** 2026-08-19T17:40:00Z **Origin:** post-BOB-126 chain-close follow-up. §11.4.263 landed as universal constitution anchor but its recommended gates (CM-KILLPG-PGID-GUARD, CM-TEST-MOCK-PID-EXPLICIT-INT) were declared “gate-code = separate work item” per §11.4.6. This plan implements those gates + fixes one pre-existing test cleanup RuntimeWarning surfaced by the post-BOB-126 verification sweep.

Context

The BOB-126 root cause was `os.killpg(1, SIGKILL) = kill(-1, SIGKILL)` due to `MagicMock.__int__==1` default. §11.4.263 mandates:

- **(A)** never signal `pgid ≤ 1`
- **(B)** validate `isinstance(pid, int)` and `pid > 1` before every process-group signal call
- **(C)** tests mocking subprocess/proc objects MUST set `mock.pid` explicitly as `int`

The universal anchor is landed in constitution 502586c. The MECHANICAL enforcement gates are recommended but not yet coded. This plan codes them for boba, following §11.4.4(b) four-layer coverage per gate.

Global Constraints

1. **§11.4.6 anti-bluff:** every gate MUST be self-validating — a golden-good fixture PASSES, a golden-bad fixture FAILS. Both wired into a paired §1.1 mutation test.

2. **§11.4.115 RED-first:** for each gate, the paired mutation MUST be observed to make the gate FAIL before the gate is trusted.
3. **§11.4.183 host-safety:** no test that could kill the host. All process-group signal tests MUST mock `os.killpg` explicitly per §11.4.263(C).
4. **§11.4.108 four-layer verification:** SOURCE → ARTIFACT → RUNTIME-ON-CLEAN-TARGET → USER-VISIBLE. Each gate has runtime evidence captured.
5. Bash gates live in `scripts/pre_build/` with the naming convention `check_cm_<slug>.sh`; they exit 0 on GREEN, non-zero on FAIL, and print a machine-parseable one-line status.
6. Every new gate script MUST have companion doc in `docs/scripts/<name>.md` per §11.4.18.
7. Tests live in `tests/pre_build/` mirroring the script name.
8. Commits are scoped: one commit per task, small + descriptive, per Conventional Commits format.

Task 1: Implement CM-KILLPG-PGID-GUARD pre-build gate

Purpose: statically detect calls to `os.killpg(...)`, `os.kill(-pid, ...)`, `subprocess.killpg`, `Process.killpg`, `killpg` (in bash — that are NOT preceded by an `isinstance(pid, int)` and `pid > 1` guard within 10 lines. Report each violation with file+line.

Deliverables:

- `scripts/pre_build/check_cm_killpg_pgid_guard.sh` — the gate script
 - Scope: scan `download-proxy/src/`, `plugins/`, `scripts/` (excluding tests + submodules)
 - Detection: grep for `os\killpg\`(, `os\kill\`(\s*- in Python; `killpg\s`, `kill\s+`-\d in bash
 - For each hit: check the 10 preceding lines for `isinstance\`(. *, `int\`) and `> 1` on a matching identifier
 - GREEN if 0 unguarded hits; FAIL with per-hit report otherwise
- `tests/pre_build/test_check_cm_killpg_pgid_guard.sh` — the meta-test (§1.1 mutation)
 - Golden-good fixture: a temp Python file with `os.killpg(pgid, SIGKILL)` preceded by `isinstance(_pid, int)` and `_pid > 1` guard — gate MUST PASS
 - Golden-bad fixture: a temp Python file with `os.killpg(1, SIGKILL)` with no guard — gate MUST FAIL
 - Golden-bad fixture 2: a temp Python file with `os.killpg(pgid, SIGKILL)` with a guard that says `> 0` (not `> 1`) — gate MUST FAIL (`pgid=1` slips through)

- docs/scripts/check_cm_killpg_pgid_guard.md — companion doc per §11.4.18

Acceptance criteria:

1. Gate PASSES against current boba tree (BOB-126 fix landed the guard in search.py:1200-1215)
2. Meta-test PASSES with golden-good and both golden-bad fixtures behaving per spec
3. Doc has: Purpose / Usage / Inputs / Outputs / Side-effects / Dependencies / Cross-references (§11.4.263)

Task 2: Implement CM-TEST-MOCK-PID-EXPLICIT-INT pre-build gate

Purpose: statically detect test files that create AsyncMock() / MagicMock() for subprocess-like objects (identified by .stdout, .stderr, .wait, or .pid attribute references within 20 lines) — but do NOT explicitly set .pid = <int> on that mock.

Deliverables:

- scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh — the gate script
 - Scope: scan tests/**/*.py
 - Detection: for each AsyncMock() or MagicMock() assignment, look forward 20 lines for subprocess-like usage indicators (.stdout.readline, .stderr.read, .wait, .returncode, .kill)
 - If subprocess-like AND no <var>.pid = <int> assignment found within 20 lines, flag as violation
 - Special case: skip if os.killpg and os.getpgid are both patched within 30 lines (either belt-and-suspenders means the mock.pid can be MagicMock safely)
- tests/pre_build/test_check_cm_test_mock_pid_explicit_int.sh — the meta-test
 - Golden-good fixture: AsyncMock() with mock.pid = 12345 — gate MUST PASS
 - Golden-good fixture 2: AsyncMock() with subprocess usage + BOTH patch.object(os, 'killpg') and patch.object(os, 'getpgid') — gate MUST PASS
 - Golden-bad fixture: AsyncMock() with subprocess usage, no mock.pid = int, no killpg patch — gate MUST FAIL
- docs/scripts/check_cm_test_mock_pid_explicit_int.md — companion doc

Acceptance criteria:

1. Gate PASSES against current boba tree (BOB-126 fix landed both mock.pid = 12345 AND patch.object(os, 'killpg') in test_deadline_hit_flag_true_when_readline_times_out)

2. Meta-test PASSES with all 3 fixtures behaving per spec

Task 3: Fix RuntimeError “coroutine ‘_bounded’ was never awaited”

Purpose: the post-BOB-126 verification sweep surfaced 2 test cases raising `RuntimeWarning: coroutine 'SearchOrchestrator._run_search.<locals>._bounded' was never awaited` in `tests/unit/merge_service/test_search_deep_coverage.py` (`TestRunSearchDiagClear::test_run_search_clears_stale_diag`, `TestRunSearchStatSeed::test_run_search_backfills_missing_stats`). This is a test-cleanup issue — the tests trigger `_run_search` but don’t await the inner `_bounded` coroutine before teardown.

Deliverables:

- Fix the 2 test cases so `_bounded` is either awaited to completion OR properly cancelled + awaited to consume the exception. Zero `RuntimeWarnings`.
- No functional change to production code.

Acceptance criteria:

1. The 2 specific tests still PASS
2. No `RuntimeWarning: coroutine '_bounded' was never awaited` surfaces in the full `merge_service` unit sweep
3. Test cleanup discipline preserved — no dangling tasks after teardown

Sequencing

Tasks 1, 2, 3 are independent and can be dispatched in any order. Recommended: Task 1 first (highest-impact gate), then Task 2 (adjacent), then Task 3 (cleanup).